

4.1.

Problems with Ball.java:

1. getVolume() returned 0.
2. getColor() returned null.
3. Ball(double volume, Color color) constructor assigns parameter variables to new variables of the same name.

Fixes:

1. Replaced return 0 in getVolume() with return volume.
2. Replaced return null in getColor() to return color.
3. Changed the name of the parameters in Ball(double volume, Color color) so that the constructor now has parameters of "volume1" and "color1". These were then assigned to new variables "volume" and "color" respectively.

4.2

There are two obvious approaches to implementing getVolume():

1. **Every time getVolume() is called, go through all the Balls in the Set and add up the volumes. Hint: one solution might be to use a for-each loop to extract Balls from the Set.**
2. **Keep track of the total volume of the Balls in BallContainer whenever Balls are added and removed. This eliminates the need to perform any computations when getVolume() is called.**

Which approach do you think is the better one? Why?

- I believe that the better approach would be the first approach. I came to this conclusion because I figured that the second approach was less efficient. Let's say we have a situation where 2 balls are added to the set. In this case, the volume will either be added to a total volume variable every time a new ball is created, or the total volume will be found once the getVolume() method is called. In this scenario, it would take roughly the same computing time. However, if we look at a situation where 3 balls are added, 1 ball is removed, 2 balls are added, 1 ball is removed, followed by calling getVolume(), the first approach would be better. This is because when getVolume() is called, only 3 balls would have their volume summed. However, if we were to use approach 2, we would have 5 addition operations and 2 subtraction operations, which would take more computing time.

4.3

1. **There are many ways to implement getBallsFromSmallest(). Briefly describe at least two different ways. Your answers should differ in the implementation of Box, not in lower-level implementation (for example, using an insertion sort instead of a selection sort is a lower-level implementation because it does not affect how Box is implemented). Hint: think about different places in the Box class where you could add code to achieve the desired functionality.**
 - The first implementation of getBallsFromSmallest() is to create an arrayList and put all the ball objects inside of it. Next, I created a comparator which when called within a sort() function would sort the getBallsFromSmallest(). Finally, it would return the iterator of the new arrayList. The second implementation I considered was one where I created a new arrayList and placed them in the arrayList in the

correct order from the start by comparing the ball volumes against all other ball volumes in the ballContainer. Then, I would return the iterator of the new arrayList. This means that all code needed for this function would be handled within the function and not be called from another portion of the Box class.

2. Which of the above ways do you think is the best? Why?

- I thought the first implementation was better because it relied more on pre-made algorithms and functions. This would be an example of “Know and Use the Libraries”, instead of putting more work on myself implementing the code.